SIMATS ENGINEERING

Short Answer Test

COURSE NAME: Software Testing **COURSE CODE: CSA3712**

COURSE OUTCOME COVERED

CO1: *Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.*

Assessment Tool Weightage: 30%

Sl. No	Questions	Bloom's Level
1	A startup team relies on random testing methods, leading to inconsistent results and missed defects. The project manager wants to improve testing effectiveness. As a test engineer, apply the concept of testing as an engineering activity to redesign the approach. Explain how structured test design improves quality and defect detection.	3
2	A banking system undergoes multiple failures due to lack of a defined testing process. The organization plans to standardize its testing activities. Illustrate how a well-defined testing process improves software quality in such real-time systems. Explain the benefits with relevant examples.	3
3	During development of an e-commerce platform, testing activities are performed inconsistently across phases. This leads to integration and system-level defects. Demonstrate how testing should be carried out as a structured process in SDLC. Explain how process discipline ensures better quality outcomes.	3
4	A login system shows mismatch between expected and actual outputs during validation. The development team is unsure whether the issue is an error, defect, or failure. Apply basic testing definitions to classify the issue. Explain the relationship between these terms with suitable examples.	3
5	A team struggles to detect defects early, leading to increased cost during later stages. The manager wants to improve defect detection efficiency. Apply a software testing principle to address this issue. Explain how the principle improves testing effectiveness with an example.	3
6	During requirement analysis of a healthcare system, several ambiguities go unnoticed. Later, these lead to defects in implementation. Show how a tester can contribute during the requirement phase to improve quality. Explain the importance of early testing involvement.	3
7	A payroll system produces incorrect outputs due to unclear and ambiguous requirements. The development team needs to identify the origin of defects. Analyze how such defects originate in the software lifecycle. Explain how requirement clarity can prevent such issues.	3

8	A system crashes when boundary input values are entered by users. The team suspects improper testing coverage. Classify this defect and justify your answer. Explain how boundary value analysis could have prevented this issue.	3
9	A company maintains a defect repository but rarely uses it for improving testing practices. This results in repeated defects across releases. Demonstrate how a defect repository can support better test case design. Explain its role in improving software quality.	3
10	An ATM system fails during withdrawal due to incorrect handling of transaction logic. The issue was not detected during testing. Apply your testing knowledge to identify and describe this as a functional defect. Explain how it could have been prevented.	3
11	A large application shows that most defects are concentrated in a few modules. The testing team wants to optimize their efforts. Use the concept of defect clustering to identify testing priorities. Explain how this improves testing efficiency.	3
12	A web application experiences slow performance under heavy user load. The issue needs to be categorized properly. Apply defect classification techniques to identify this type of defect. Explain how classification helps in better resolution.	3
13	A team is preparing for regression testing after multiple releases. They have access to historical defect data. Show how this data can be reused effectively for regression testing. Explain how it improves test coverage and efficiency.	3
14	During integration testing, multiple defects are found due to poor communication between developers and testers. The organization wants to improve collaboration. Identify the role of a tester in minimizing such defects. Explain how early involvement helps.	3
15	A software product repeatedly faces similar defects in every release cycle. The team needs a long-term solution. Apply defect prevention techniques to address recurring issues. Explain how prevention improves overall software quality.	3
16	A mobile application frequently crashes due to inconsistent testing approaches. The testing team wants to standardize practices. Apply testing principles to redesign the testing strategy. Explain how structured approaches improve reliability.	3
17	A company confuses debugging with testing, leading to delays in defect resolution. The team needs clarity on both concepts. Differentiate testing and debugging using a real-time scenario. Explain their roles in improving software quality.	3
18	An online shopping system behaves differently across browsers. This leads to customer dissatisfaction. Apply testing principles to design a cross-browser testing strategy. Explain how such issues can be effectively handled.	3

19	A logistics application produces inconsistent outputs due to improper validation. The issue is not detected early. Apply testing concepts to identify gaps in validation. Explain how proper testing improves system reliability.	3
20	A team skips structured test case design and directly executes random tests. This leads to poor defect detection. Apply testing as an engineering activity to improve the process. Explain the importance of systematic test design.	3
21	A healthcare system shows data inconsistency between modules due to integration issues. The testing team needs to analyze the problem. Apply testing principles to identify root causes. Explain how proper testing ensures data consistency.	3
22	A project experiences delays because testers are not involved early in development. The organization wants to improve collaboration. Explain the role of testers across SDLC phases. Show how early involvement improves quality.	3
23	A system fails during peak usage due to untested scenarios. The testing team missed critical cases. Apply testing principles to identify gaps. Explain how better test coverage prevents such failures.	3
24	A software team lacks proper defect documentation, making tracking difficult. The organization wants to improve defect management. Apply defect repository concepts to solve this issue. Explain how documentation enhances tracking and analysis.	3
25	A banking system produces repeated transaction failures due to improper testing practices. The team needs to improve quality. Apply software testing principles to redesign the testing approach. Explain how structured testing enhances reliability.	3
26	A system shows failures due to missing edge case testing. The testers focused only on normal inputs. Apply testing principles to include edge cases. Explain how boundary testing improves defect detection.	3
27	A development team struggles to maintain consistency in testing across modules. This leads to uneven quality. Apply process-based testing approaches to address this issue. Explain how standardization improves outcomes.	3
28	A web application fails due to improper handling of unexpected inputs. The testing team missed validation scenarios. Apply testing concepts to improve input validation. Explain how proper testing ensures robustness.	3
29	A software company faces frequent defect leakage into production. The management wants to reduce such issues. Apply testing principles to analyze the root cause. Explain how improved testing practices prevent leakage.	3
30	A project team is unable to identify defects early due to poor testing strategies. This increases overall cost and effort. Apply testing principles to redesign the strategy. Explain how early defect detection improves efficiency.	3

Code of Conduct:

I M Ravi Sai Vinay (192311035) *certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:M Ravi Sai Vinay

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	**30**	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	**25**	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	**20**	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	**15**	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	**10**	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand

SHORT ANSWERS

1. Structured test design uses systematic techniques, clear test cases, and coverage criteria, improving consistency and detecting more defects.

2. A well-defined testing process standardizes activities, improves reliability, and prevents failures through planned testing at each stage.

3. Testing should be performed systematically throughout the SDLC, ensuring early detection, proper integration, and consistent quality.

4. An error is a human mistake, a defect is the resulting flaw in software, and a failure occurs when that defect causes incorrect behavior during execution.

5. Early testing helps detect defects sooner, reducing correction costs and improving overall testing effectiveness.

6. Testers can review requirements for ambiguity, completeness, and correctness, preventing defects before implementation begins.

7. Unclear requirements can introduce defects during design and coding, while clear, validated requirements reduce misunderstandings and errors.

8. This is a boundary-related defect, and Boundary Value Analysis tests values at and around limits to detect such failures.

9. A defect repository records past defects and helps testers create targeted test cases to prevent similar defects in future releases.

10. The ATM issue is a functional defect because transaction logic produces incorrect behavior; functional test cases could have detected it.

11. Defect clustering means focusing more testing on modules with high defect concentration, improving testing efficiency and coverage.

12. Slow performance under heavy load is a performance defect, and classification helps testers select appropriate performance-testing techniques.

13. Historical defect data helps identify high-risk areas and create focused regression tests, improving coverage and reducing testing effort.

14. Early tester involvement improves communication, identifies integration risks early, and prevents defects caused by misunderstandings.

15. Defect prevention uses root-cause analysis, process improvement, reviews, and standards to stop recurring defects from reappearing.

16. A standardized testing strategy with defined processes, test cases, reviews, and coverage criteria improves mobile application reliability.

17. Testing identifies and reports defects, while debugging locates and fixes their causes; both together improve software quality.

18. Cross-browser testing should verify functionality, layout, and compatibility across major browsers and versions to prevent browser-specific defects.

19. Proper validation using requirements, input checks, and systematic test cases identifies inconsistencies early and improves system reliability.

20. Systematic test design uses requirements, test techniques, and coverage criteria instead of random testing, improving defect detection.

21. Integration testing with interface checks, data validation, and end-to-end scenarios helps identify root causes and maintain data consistency.

22. Testers should participate from requirements through maintenance, enabling early defect detection, better communication, and improved quality.

23. Better test coverage includes peak-load, stress, boundary, and realistic usage scenarios, helping prevent failures during heavy usage.

24. A defect repository should record defect details, severity, status, cause, and resolution, enabling effective tracking and analysis.

25. Structured testing with planned test cases, reviews, regression testing, and proper coverage reduces transaction failures and improves reliability.

26. Boundary testing checks minimum, maximum, and just-inside/outside values, helping detect defects missed by normal-input testing.

27. Process-based testing standardizes planning, design, execution, reporting, and review across modules, ensuring consistent quality.

28. Testing should include valid, invalid, unexpected, and boundary inputs to verify validation and improve application robustness.

29. Production defect leakage can be reduced through risk-based testing, reviews, automation, regression testing, and better test coverage.

30. Early defect detection through reviews, unit testing, integration testing, and continuous testing reduces rework, cost, and overall effort.